

Chapter 8 Summary

Summary in Arabic

المخطط الذهني للفصل الثامن: الجبر العلائقي وحساب العلائق

الجبر العلائقي وحساب العلائق

- مقدمة إلى الجبر العلائقي 1.
 - تعريف الجبر العلائقي
 - خاصية الإغلاق (Closure)
 - التعبيرات الجبرية العلائقية
- العمليات الجبرية العلائقية 2.
 - أ. العمليات الأحادية (Unary Operations)
 - SELECT (σ)
 - PROJECT (π)
 - RENAME (ρ)
 - ب. عمليات نظرية المجموعات (Set Theory Operations)
 - UNION ($\cup$)
 - INTERSECTION ($\cap$)
 - SET DIFFERENCE ($-$)
 - CARTESIAN PRODUCT ($\times$)
 - ج. العمليات الثنائية (Binary Operations)
 - JOIN ($\bowtie$)
 - Theta Join
 - Equijoin
 - Natural Join ($*$)
 - DIVISION ($\div$)
 - د. عمليات إضافية (Additional Operations)
 - Aggregate Functions (SUM, AVG, COUNT, MAX, MIN)
 - Grouping with Aggregation
 - Outer Joins (Left, Right, Full)
 - Outer Union
 - Recursive Closure (غير مدعومة في الجبر العلائقي الأساسي)
- أمثلة تطبيقية على استعلامات الجبر العلائقي 3.
 - أمثلة خطوة بخطوة

- كتابة التعبيرات كتعبير واحد أو عبر نتائج وسيطة
- 4. شجرة الاستعلام (Query Tree)
 - تمثيل بصري للتعبيرات الجبرية لأغراض التحسين
- 5. مقدمة إلى حساب العلائق (Relational Calculus)
 - Tuple Relational Calculus
 - Domain Relational Calculus

التلخيص الشامل بالتفصيل

1. مقدمة إلى الجبر العلائقي

- الجبر العلائقي (Relational Algebra) هو مجموعة من العمليات الرياضية التي تُستخدم لاسترجاع البيانات من قواعد البيانات العلائقية.
- كل عملية في الجبر العلائقي تأخذ علاقة (Relation) كمدخل (أو أكثر) وتُنتج علاقة جديدة كمخرج.
- هذه الخاصية تُعرف باسم الإغلاق (Closure)، لأن كل كيان في الجبر العلائقي هو علاقة.
- يمكن تسلسل العمليات لتكوين تعبير جبري علائقي (Relational Algebra Expression)، والذي يمثل استعلامًا كاملاً.

💡 ملاحظة: الجبر العلائقي لغة إجرائية (Procedural)، أي أن المستخدم يحدد كيف يتم الحصول على النتيجة، وليس فقط ما هي النتيجة.

2. العمليات الجبرية العلائقية

أ. العمليات الأحادية (Unary Operations)

تُطبّق على علاقة واحدة.

1. SELECT – الرمز: σ (سيجما)

- تُستخدم لـ تصفية السجلات (الصفوف) بناءً على شرط منطقي.
- الشكل العام:
 $\sigma_{\langle \text{condition} \rangle}(R)$
- خصائص:
 - تحافظ على نفس مخطط العلاقة الأصلية.
 - تبادلية:
 $\sigma_{\langle \text{cond1} \rangle}(\sigma_{\langle \text{cond2} \rangle}(R)) = \sigma_{\langle \text{cond2} \rangle}(\sigma_{\langle \text{cond1} \rangle}(R))$
- يمكن دمج تسلسل من عمليات SELECT إلى شرط واحد باستخدام AND.
- عدد الصفوف في الناتج $\geq$ عدد الصفوف في المدخل.

مثال: ✓

- $\sigma \text{ DNO}=4(\text{EMPLOYEE}) \rightarrow$ جميع الموظفين في القسم رقم 4.
- $\sigma \text{ SALARY} > 30000(\text{EMPLOYEE}) \rightarrow$ الموظفون الذين رواتبهم أكثر من 30,000.

2. PROJECT – π (باي)

- تُستخدم لـ اختيار أعمدة معينة (السمات) من العلاقة، وحذف البقية.
- الشكل العام:
 $\pi \langle \text{attribute list} \rangle (R)$
- تُحذف التكرارات تلقائيًا لأن العلاقة هي مجموعة رياضية (لا تسمح بالتكرار).
- خصائص:
- عدد الصفوف في الناتج $\geq$ عدد الصفوف في المدخل.
- إذا تضمن القائمة مفتاحًا أساسيًا، فلن يُحذف أي صف.
- ليست تبادلية.

مثال: ✓

$\pi \text{ LNAME, FNAME, SALARY}(\text{EMPLOYEE}) \rightarrow$ أسماء الموظفين ورواتبهم فقط.

3. RENAME – ρ (رو)

- تُستخدم لتغيير اسم العلاقة أو أسماء السمات (الأعمدة).
- الأشكال:
- $\rho S(R) \rightarrow$ تغيير اسم العلاقة فقط.
- $\rho(B_1, B_2, \dots, B_n)(R) \rightarrow$ تغيير أسماء السمات فقط.
- $\rho S(B_1, \dots, B_n)(R) \rightarrow$ تغيير كليهما.

💡 ملاحظة: ضرورة عند الحاجة إلى ربط علاقة بنفسها (Self-Join) أو عند تجنب التضارب في أسماء السمات.

ب. عمليات نظرية المجموعات

1. UNION ($\cup$)

- تُنتج جميع الصفوف الموجودة في إحدى العلاقتين أو كليهما.
- شروط التوافق (Type Compatibility):
- نفس عدد السمات.
- نفس نوع البيانات لكل سمة متقابلة.
- تُحذف التكرارات.

مثال: ✓

```
RESULT1  $\leftarrow \pi \text{ SSN}(\sigma \text{ DNO}=5(\text{EMPLOYEE}))$ 
RESULT2  $\leftarrow \pi \text{ SUPERSSN}(\sigma \text{ DNO}=5(\text{EMPLOYEE}))$ 
RESULT  $\leftarrow \text{RESULT1} \cup \text{RESULT2}$ 
```

$\rightarrow$ أرقام الضمان الاجتماعي لكل من يعمل في القسم 5 أو يشرف على موظف فيه.

2. INTERSECTION ($\cap$)

- تُنتج الصفوف المشتركة بين العلاقتين.
- تتطلب نفس شروط التوافق.

3. SET DIFFERENCE ($-$)

- تُنتج الصفوف الموجودة في العلاقة الأولى فقط وليس في الثانية.
- غير تبادلية: $R - S \neq S - R$.

4. CARTESIAN PRODUCT ($\times$)

- تُنتج جميع التوافق الممكنة من صفوف العلاقتين.
- لا تتطلب التوافق بين العلاقتين.
- الناتج يحتوي على $|R| \times |S|$ صفًا.
- عادةً غير ذات معنى لوحدها، لكنها تصبح مفيدة عند دمجها مع SELECT.

✓ مثال تطبيقي:

```
FEMALE_EMPS  $\leftarrow$   $\sigma$  SEX='F'(EMPLOYEE)
EMP_NAMES  $\leftarrow$   $\pi$  FNAME, LNAME, SSN(FEMALE_EMPS)
EMP_DEPENDENTS  $\leftarrow$  EMP_NAMES  $\times$  DEPENDENT
ACTUAL_DEPS  $\leftarrow$   $\sigma$  SSN=ESSN(EMP_DEPENDENTS)
```

→ الآن لدينا فقط التبعية الحقيقية للموظفات.

ج. العمليات الثنائية

1. JOIN ($\bowtie$)

- تجميع عمليتي **CARTESIAN PRODUCT + SELECT** في خطوة واحدة.
- الشكل العام:
 $R \bowtie \langle \text{condition} \rangle S$
- الناتج يحتوي فقط على الصفوف التي تحقق شرط الربط.

أنواع JOIN:

- **Theta Join:** (قد يحتوي على $=, <, >, \neq, \dots$)
- **Equijoin:** شرط يحتوي فقط على مساواة ($=$) بين سمات.
يحتفظ بكلتا السمتين في الناتج (مثلاً: DNUMBER من DEPARTMENT و DNO من EMPLOYEE).
- **Natural Join (*):**
 - يُجرى تلقائيًا على السمات المشتركة ذات الأسماء المتطابقة.
 - يحتفظ بسمة واحدة فقط من كل زوج متطابق.

✓ مثال Natural Join:

```
DEPT_LOCS  $\leftarrow$  DEPARTMENT * DEPT_LOCATIONS
```

→ يربط الجداول على DNUMBER تلقائيًا.

2. DIVISION (÷)

- من أصعب العمليات فهمًا.
- تُستخدم للإجابة عن أسئلة مثل:
"من هم الموظفون الذين لديهم تبعية في كل مشروع؟"
- الشكل: $R(Z) \div S(X)$ حيث $X \subseteq Z$.
- الناتج: سمات $Y = Z - X$ ، بحيث لكل قيمة في Y ، توجد جميع القيم في S .

📌 مثال بسيط:

إذا كانت $R(A, B)$ و $S(B)$ ، فإن $R \div S$ يعطي قيم A التي تظهر مع كل قيمة في S .

د. العمليات الإضافية

1. دوال التجميع (Aggregate Functions)

- لا يمكن التعبير عنها بالجبر العلائقي الأساسي.
- تشمل:

SUM , AVG , MAX , MIN , COUNT

- الرمز: $\mathcal{F}$ (Script F)

✅ أمثلة:

$\mathcal{F}$ MAX Salary(EMPLOYEE) → أعلى راتب.

$\mathcal{F}$ COUNT SSN, AVG Salary(EMPLOYEE) → عدد الموظفين ومتوسط الراتب.

2. التجميع حسب سمة (Grouping)

- يُستخدم مع دوال التجميع.
- الشكل: $\mathcal{F} \langle \text{aggregate functions} \rangle (R) \langle \text{grouping attrs} \rangle$

✅ مثال:

DNO $\mathcal{F}$ COUNT SSN, AVG Salary(EMPLOYEE)

→ لكل قسم: عدد الموظفين ومتوسط الراتب.

3. الربط الخارجي (Outer Joins)

- يحل مشكلة فقدان السجلات التي ليس لها مقابل في الطرف الآخر.
- الأنواع:
- **Left Outer Join**: يحتفظ بجميع سجلات العلاقة اليسرى.
- **Right Outer Join**: يحتفظ بجميع سجلات العلاقة اليمنى.
- **Full Outer Join**: NULL يحتفظ بجميع السجلات من الطرفين، مع ملء القيم المفقودة بـ.

4. الاتحاد الخارجي (Outer Union)

- يُستخدم عندما تكون العلاقتان غير متوافقتين تمامًا، لكن لديهما سمات مشتركة جزئيًا.
- يدمج السمات المشتركة، ويحتفظ ببقية السمات من كلا العلاقتين مع وضع **NULL** عند الضرورة.

مثال: ✓

```
STUDENT(Name, SSN, Dept, Advisor)
INSTRUCTOR(Name, SSN, Dept, Rank)
```

→ الناتج: Name, SSN, Dept, Advisor, Rank

5. الإغلاق التكراري (Recursive Closure)

- غير مدعوم في الجبر العلائقي الأساسي.
- مثال: "أوجد جميع الرؤوسيين لمدير معين بجميع المستويات".
- يتطلب آليات تكرارية (مثل **WITH RECURSIVE** في SQL3).

3. أمثلة تطبيقية على استعلامات الجبر العلائقي

مثال Q1:

"استرجع اسم وعنوان جميع الموظفين العاملين في قسم 'Research'."

- الطريقة الإجرائية (خطوة بخطوة):

```
RESEARCH_DEPT ← σ DNAME='Research' (DEPARTMENT)
RESEARCH_EMPS ← RESEARCH_DEPT ⋈ DNUMBER=DNO EMPLOYEE
RESULT ← π FNAME, LNAME, ADDRESS(RESEARCH_EMPS)
```

- كتعبير واحد:

```
π FNAME, LNAME, ADDRESS(σ DNAME='Research' (DEPARTMENT ⋈ DNUMBER=DNO
EMPLOYEE))
```

مثال Q6:

"استرجع أسماء الموظفين الذين ليس لديهم تبعيات."

- الحل:

```
ALL_EMPS ← π SSN(EMPLOYEE)
EMPS_WITH_DEPS ← π ESSN(DEPENDENT)
EMPS_WITHOUT_DEPS ← ALL_EMPS - EMPS_WITH_DEPS
RESULT ← π LNAME, FNAME(EMPS_WITHOUT_DEPS * EMPLOYEE)
```

💡 ملاحظة مهمة: استخدام * (Natural Join) هنا آمن لأن كلا العلاقتين تحتويان على SSN.

4. شجرة الاستعلام (Query Tree)

- هي تمثيل بياني للتعبير الجبري.
- العقد الطرفية (Leaves): تمثل العلاقات الأساسية (مثل EMPLOYEE).
- العقد الداخلية: تمثل العمليات (مثل SELECT, PROJECT, JOIN).
- تُستخدم في تحسين الاستعلامات (Query Optimization) لاختيار أفضل ترتيب للتنفيذ.

5. مقدمة إلى حساب العلائق (Relational Calculus)

- لغة غير إجرائية (Non-Procedural).
- المستخدم يصف ما يريد، وليس كيف يحصل عليه.
- نوعان رئيسيان:
 1. حساب العلائق على أساس الصف (Tuple Relational Calculus):
مثل: $\{ t \mid \text{EMPLOYEE}(t) \text{ AND } t.\text{Salary} > 50000 \}$
 2. حساب العلائق على أساس المجال (Domain Relational Calculus):
يعمل على قيم السمات (المجالات) بدلاً من الصفوف.

🔴 ملاحظة نهائية: الجبر العلائقي وحساب العلائق مكافئان من حيث القوة التعبيرية، أي أن أي استعلام يمكن كتابته بأحدهما يمكن كتابته بالآخر.

الخلاصة النهائية

الفصل الثامن يقدم أساسيات معالجة الاستعلامات في قواعد البيانات العلائقية عبر:

- الجبر العلائقي: لغة إجرائية تعتمد على عمليات واضحة.
- حساب العلائق: لغة وصفية تعتمد على المنطق.

العمليات الأساسية تشمل: الانتقاء، الإسقاط، الاتحاد، الطرح، الضرب الديكارتي، والربط، مع توسعات مهمة مثل التجميع، الربط الخارجي، والقسم.

الفهم العميق لهذه المفاهيم ضروري لتصميم استعلامات فعّالة، وفهم كيفية عمل محركات قواعد البيانات داخليًا.

Summary in English

Mind Map: Chapter 8 – The Relational Algebra and Relational Calculus

The Relational Algebra and Relational Calculus

- └─ 1. Introduction to Relational Algebra
 - └─ Definition & Purpose
 - └─ Closure Property
 - └─ Relational Algebra Expressions
- └─ 2. Relational Algebra Operations
 - └─ A. Unary Operations
 - └─ SELECT (σ)
 - └─ PROJECT (π)
 - └─ RENAME (ρ)
 - └─ B. Set-Theoretic Operations
 - └─ UNION ($\cup$)
 - └─ INTERSECTION ($\cap$)
 - └─ SET DIFFERENCE ($-$)
 - └─ CARTESIAN PRODUCT ($\times$)
 - └─ C. Binary Operations
 - └─ JOIN ($\bowtie$)
 - └─ Theta Join
 - └─ Equijoin
 - └─ Natural Join ($*$)
 - └─ DIVISION ($\div$)
 - └─ D. Additional Operations
 - └─ Aggregate Functions (SUM, AVG, COUNT, MAX, MIN)
 - └─ Grouping with Aggregation
 - └─ Outer Joins (Left, Right, Full)
 - └─ Outer Union
 - └─ Recursive Closure (Not supported in basic algebra)
- └─ 3. Query Examples in Relational Algebra
 - └─ Step-by-step procedural form
 - └─ Single-expression form
- └─ 4. Query Trees
 - └─ Visual representation for optimization
- └─ 5. Introduction to Relational Calculus
 - └─ Tuple Relational Calculus
 - └─ Domain Relational Calculus



1. Introduction to Relational Algebra

- **Relational Algebra** is a procedural query language that consists of a set of operations used to manipulate relations (tables) in the relational model.
- Every operation takes **one or more relations as input** and **produces a new relation as output**.
- This is known as the **closure property**: all objects in relational algebra are relations.
- A **relational algebra expression** is a sequence of operations that represents a database query.
- It is **procedural**: the user specifies **how** to retrieve the data, not just **what** data is needed.

💡 **Historical Note:** The word "algebra" comes from the Arabic *al-jabr*, from the book by **Muhammad ibn Musa al-Khwarizmi** (800–847 CE). The word "algorithm" also derives from his name.

2. Relational Algebra Operations

A. Unary Operations (operate on a single relation)

1. SELECT – σ (sigma)

- Filters tuples (rows) based on a **Boolean condition**.
- Syntax: $\sigma_{\langle \text{condition} \rangle}(R)$
- **Properties:**
 - Output schema = input schema.
 - **Commutative:** $\sigma_{\langle \text{cond1} \rangle}(\sigma_{\langle \text{cond2} \rangle}(R)) = \sigma_{\langle \text{cond2} \rangle}(\sigma_{\langle \text{cond1} \rangle}(R))$
 - Cascaded SELECTs can be combined:
 $\sigma_{\langle \text{cond1} \rangle}(\sigma_{\langle \text{cond2} \rangle}(R)) = \sigma_{\langle \text{cond1} \wedge \text{cond2} \rangle}(R)$
 - Result size $\leq$ input size.

✅ Example:

$\sigma_{\text{DNO}=4}(\text{EMPLOYEE}) \rightarrow$ Employees in department 4.

$\sigma_{\text{SALARY} > 30000}(\text{EMPLOYEE}) \rightarrow$ Employees earning more than \$30,000.

2. PROJECT – π (pi)

- Selects specific **attributes (columns)** and discards others.
- Syntax: $\pi_{\langle \text{attribute list} \rangle}(R)$
- **Removes duplicate tuples** (relations are sets).
- **Not commutative.**
- If the attribute list includes a **key**, no duplicates are lost.

✓ **Example:**

π FNAME, LNAME, SALARY(EMPLOYEE) $\rightarrow$ First name, last name, and salary of all employees.

3. RENAME – ρ (rho)

- Used to rename a relation or its attributes.
- Forms:
 - $\rho S(R)$ $\rightarrow$ rename relation to S.
 - $\rho(B_1, \dots, B_n)(R)$ $\rightarrow$ rename attributes.
 - $\rho S(B_1, \dots, B_n)(R)$ $\rightarrow$ rename both.

💡 **Why needed?** Essential for self-joins or avoiding attribute name conflicts.

B. Set-Theoretic Operations

Require **type compatibility**: same number of attributes, and corresponding attributes have compatible domains.

1. UNION ($\cup$)

- Returns all tuples in **R, S, or both**.
- Duplicates removed.

2. INTERSECTION ($\cap$)

- Returns tuples **common to both R and S**.

3. SET DIFFERENCE ($-$)

- Returns tuples in **R but not in S**.
- **Not commutative**: $R - S \neq S - R$.

4. CARTESIAN PRODUCT ($\times$)

- Combines **every tuple of R with every tuple of S**.
- Result size = $|R| \times |S|$.
- Not meaningful alone; usually followed by **SELECT**.

✓ **Example:**

To find female employees and their dependents:

```
FEMALE_EMPS  $\leftarrow$   $\sigma$  SEX='F'(EMPLOYEE)
EMP_NAMES  $\leftarrow$   $\pi$  FNAME, LNAME, SSN(FEMALE_EMPS)
```

```
EMP_DEPENDENTS ← EMPNAMES × DEPENDENT
ACTUAL_DEPS ←  $\sigma$  SSN=ESSN(EMP_DEPENDENTS)
RESULT ←  $\pi$  FNAME, LNAME, DEPENDENT_NAME(ACTUAL_DEPS)
```

C. Binary Operations

1. JOIN ($\bowtie$)

- Combines **CARTESIAN PRODUCT + SELECT** into one step.
- General form: $R \bowtie \text{<condition>} S$
- Only **related tuples** (satisfying the condition) appear in the result.

Types of JOIN:

- **Theta Join:** General condition (e.g., $R.A < S.B$).
- **Equijoin:** Condition uses **only equality** ($=$).
→ Keeps **both** matching attributes in result.
- **Natural Join** (*):
 - Automatically joins on **attributes with same name**.
 - Keeps **only one copy** of each shared attribute.

✅ Example:

```
DEPT_LOCS ← DEPARTMENT * DEPT_LOCATIONS
```

→ Joins on **DNUMBER** (common attribute).

2. DIVISION ($\div$)

- Used for queries like:
“Find employees who work on **all** projects.”
- Formal: $R(Z) \div S(X)$, where $X \subseteq Z$.
Result = set of Y-values ($Y = Z - X$) such that **for every x in S, (y,x) is in R**.

📌 **Intuition:** If $R = (\text{Employee}, \text{Project})$, and $S = \text{all projects}$, then $R \div S = \text{employees on every project}$.

D. Additional Operations

1. Aggregate Functions

- Not part of basic algebra, but widely used.
- Functions: **SUM**, **AVG**, **MAX**, **MIN**, **COUNT**

- Notation: $\mathcal{F}$ (script F)

✓ Examples:

$\mathcal{F}$ MAX Salary(EMPLOYEE) → highest salary.

$\mathcal{F}$ COUNT SSN, AVG Salary(EMPLOYEE) → number of employees + average salary.

2. Grouping with Aggregation

- Syntax: <grouping attrs> $\mathcal{F}$ <aggregate list>(R)

✓ Example:

DNO $\mathcal{F}$ COUNT SSN, AVG Salary(EMPLOYEE)

→ For each department: employee count and average salary.

3. Outer Joins

- Preserve unmatched tuples by padding with **NULL**.
 - **Left Outer Join**: Keep all tuples from left relation.
 - **Right Outer Join**: Keep all from right.
 - **Full Outer Join**: Keep all from both.

4. Outer Union

- For **partially compatible** relations (e.g., STUDENT and INSTRUCTOR).
- Shared attributes merged; non-shared kept with NULLs.

✓ Example:

STUDENT(Name, SSN, Dept, Advisor)

INSTRUCTOR(Name, SSN, Dept, Rank)

→ Result: (Name, SSN, Dept, Advisor, Rank)

5. Recursive Closure

- **Not expressible** in basic relational algebra.
- Example: "Find all supervisees of James Borg at all levels."
- Requires recursion (supported in SQL3 via WITH RECURSIVE).

3. Query Examples

Query Q1:

"Retrieve name and address of employees in 'Research' department."

- **Procedural form:**

```
RESEARCH_DEPT ← σ DNAME='Research'(DEPARTMENT)
RESEARCH_EMPS ← RESEARCH_DEPT ⋈ DNUMBER=DNO EMPLOYEE
RESULT ← π FNAME, LNAME, ADDRESS(RESEARCH_EMPS)
```

- **Single expression:**

```
π FNAME, LNAME, ADDRESS(σ DNAME='Research'(DEPARTMENT ⋈ DNUMBER=DNO
EMPLOYEE))
```

Query Q6:

“Retrieve names of employees with no dependents.”

- **Procedural:**

```
ALL_EMPS ← π SSN(EMPLOYEE)
EMPS_WITH_DEPS ← π ESSN(DEPENDENT)
EMPS_WITHOUT_DEPS ← ALL_EMPS − EMPS_WITH_DEPS
RESULT ← π LNAME, FNAME(EMPS_WITHOUT_DEPS * EMPLOYEE)
```

- **Single expression:**

```
π LNAME, FNAME((π SSN(EMPLOYEE) − π ESSN(DEPENDENT)) * EMPLOYEE)
```

💡 Note: Natural join ($*$) works because both relations have SSN .

4. Query Trees

- A **tree representation** of a relational algebra expression.
- **Leaf nodes:** base relations (e.g., EMPLOYEE).
- **Internal nodes:** operations (e.g., σ , π , $\bowtie$).
- Used for **query optimization** (see Chapter 15).

5. Relational Calculus

- A **non-procedural** (declarative) query language.
- User specifies **what** to retrieve, not **how**.
- Two forms:
 1. **Tuple Relational Calculus (TRC):**
Uses tuple variables.
Example: $\{ t \mid \text{EMPLOYEE}(t) \wedge t.\text{Salary} > 50000 \}$
 2. **Domain Relational Calculus (DRC):**
Uses domain variables (attribute values).

✦ **Important:** Relational algebra and relational calculus are **equivalent in expressive power** (Codd's Theorem).

✓ Final Summary

Chapter 8 lays the theoretical foundation for querying relational databases:

- **Relational Algebra** provides a procedural framework with operations like SELECT, PROJECT, JOIN, etc.
- **Relational Calculus** offers a declarative alternative.
- Together, they form the basis for SQL and query processing engines.
- Advanced operations (aggregation, outer joins, grouping) extend the basic algebra for real-world needs.
- **Query trees** enable optimization, and **completeness** ensures all queries can be built from core operations.